ATTORNEY SHIELD

ASI 2.0 - Native Android & iOS

Attorney Shield 2.0 - What the Native A

Colour system: Attorney-Shield Color Scheme Review

Repository: samson-phillip/asi-claude

DRAFT FOR APPROVAL

Contents

From: mobile (Android + iOS) **Date:** 2026-08-12

The native apps now sign in against `gateway-dev` - by password **and by one-time code** - and reach the home screen. Most of what we need already exists in the schema; this is the short list of what does not, plus a few shapes we would rather confirm than guess.

---

1. Dev data is not seeded - this is what blocks us

Signed in as the test member against `https://gateway-dev.attorneyshield.io/query`:

Query	Result
`countries { iso2 }`	`[]`
`adminIncidentTypeList(activeOnly: false)`	`[]`
`adminIncidentTypeList(activeOnly: true, countryISO2: "US" \`	"GB" \
`casesByUser(userID: <test member>)`	`[]`
`adminLanguageList`	1 entry - `ar-SA`, `isDefault: false`

No errors on any of them - the data simply is not there. Because `adminIncidentTypeList` is filtered by `countryISO2` and **no countries are configured**, it returns empty for every value we tried.

The home screen therefore shows "No incident types are configured yet.", which is correct behaviour on our side but means we cannot exercise the incident tiles, the attorney chip row, or place a call with a real incident type.

We sent the same query the deployed member client sends - `adminIncidentTypeList(activeOnly: true, countryISO2: $country)`, lifted from its JS bundle - so this is not a difference in how we are asking.

What we need on dev:

1. **Countries configured**, at minimum `US`.
2. **Incident types seeded** for that country, with translations. The design reference uses: Traffic Stop, Auto Accident, Pedestrian Stop, Domestic, Test Call, Other.
3. **An English language entry**, ideally `isDefault: true`. Today the only language is `ar-SA` and it is not marked default, so even with incident types present our label resolution (English -> org default -> first -> humanized code) would fall through to Arabic or a humanized code.
4. **A case for the test member** (`munyira851@gmail.com`), so jurisdiction and partner resolve from real data instead of falling back to `DEV_DEFAULTS`. The member's organization resolves correctly already (`6c53e00d-8682-11f1-a446-06cf81ac74a7`), but with no case we cannot exercise attorney pre-selection at all.

If it is easier to point us at an environment that already has this data, that works too - we only need somewhere to develop against.

---

2. Missing from the schema entirely

Two areas returned **no matching operations**, so we cannot build them:

2.1 Situation preferences - screens 13B, 13C, 27B

The member's saved "three most common situations". Nothing matching `situation`, `preference` or `favourite` in 238 queries or 297 mutations.

Home currently shows the full incident list, because there is nowhere to store a choice of three. Roughly: a read and a write, capped at three incident-type IDs per member.

2.2 Trial and guest - screens V1-V2, T1-T8, G1-G3

Nothing matching `trial` or `guest`.

The design has a 7-day limited trial with an in-app conversion gate, and a guest mode entered from an unrecognised email at sign-in. **The question that decides the whole design: is a guest a real account with a role, or purely local state?** We would rather ask than assume.

---

3. Shapes we would rather confirm than guess

Small answers, but each one is currently a guess:

1. `countryISO2` **on** `verifyLoginOtp` - we send `null`, because it is optional and that is what the web client sends. But what is it *for*? If it affects routing or jurisdiction we would rather send something real than a null that quietly degrades.
2. **Sign-in codes are 4 digits** - confirmed against the live gateway, and we have built for 4. Flagging only because the design reference specifies a six-digit entry; we assume the reference is describing registration phone verification, which is a different code.
3. `refreshToken` - what is the access-token lifetime, and does refreshing rotate the refresh token? This matters more for us than for web: a browser tab is short-lived, but a native app sits backgrounded for days, and this is an app people open during a police encounter.
4. `setMemberPin(userId, pin)` **/** `verifyMemberPin` - the design reference says the PIN's only job is ending a live session securely; it does not unlock the app or protect recordings. Is `verifyMemberPin` the intended server-side gate for ending a call?
5. **Casing is inconsistent and we follow whatever each operation uses** - the gateway mixes `userID` (`login`, `casesByUser`) with `userId` (`setMemberPin`, `verifyMemberPin`), and comms REST uses `memberUserId`. Worth knowing before anyone adds a field.
6. **Is the one-device-at-a-time rule permanent?** `LoginPayload` returns `otherSessionsRevoked` and `mySessionStatus` reports `another_device`, so signing in on the web ends the app's session and vice versa. We can live with it, but it is worth confirming it is deliberate for a phone people open during a police encounter - if someone signs in on a laptop, the app in their pocket is signed out.

4. Deep-link contract for the web->app handoff

Screens 07 and T4 hand the member back to the app "with email pre-filled", but the design reference never records the actual path or parameter names.

Currently implemented: we accept `/app/return`, `/return-to-app` and `/app` on `attorney-shield.com` and `www.attorney-shield.com`, reading an `email` query parameter. All of it is confined to one file, so a confirmed contract is a one-line change.

We need: the real path and parameter names.

Please do not design the link to carry a credential. We treat it as untrusted input - anyone can send a link. The email is a text-field prefill only, and we have a test asserting that a link carrying `accessToken`, `userID` or `roles` yields nothing but the email. If the app needs to know what someone has bought, we would rather ask the backend after a real sign-in.

Related, and a web task rather than a backend one

For the link to open the app *silently*, both hosts need:

- **Android:** `/.well-known/assetlinks.json` with our signing-certificate SHA-256 fingerprint for `com.app.attorney.shield`
- **iOS:** `/.well-known/apple-app-site-association` for our Team ID + `com.app.attorney.shield`

Until then Android shows a "which app?" dialog and iOS universal links do not fire at all. We will send the fingerprint and Team ID once our release keystore exists.

5. One security note

`POST /api/vonage/video/member-call` **takes no authentication.**

We are not designing around it and nothing we have built depends on it staying open - but as it stands, anyone can place a call against any `organizationId`/`memberUserId` they can guess, and that call routes to a real attorney.

Flagging rather than assuming it is known.

6. What we ship as each lands

You give us	We ship
Countries, incident types, a language, and a case on dev	Home and the call flow verified against real data
Answers to §3	Token refresh, and registration screens 08-12
Situation-preference endpoints (§2.1)	The home screen's saved three, as designed
A decision on the guest model (§2.2)	Trial and guest flows scoped
The deep-link contract (§4)	A one-line change, then verified

Everything in §3 we can start immediately - the operations are in the schema and the screens are specified in the design reference.